

Multi-Speaker Bluetooth Streaming Controller

Project Handoff - 10 Independent Speakers / 500GB SSD

Objective: Replace multiple standalone MP3 players with one centrally managed device that stores the full music library and independently streams audio to up to 10 Bluetooth speakers, controlled from a phone.

Final direction	Raspberry Pi 4 (4GB) running Linux
Capacity	Up to 10 independently playing Bluetooth speakers
Storage	500GB USB SSD + small microSD for OS
Control	Custom web app accessed from phone
Audio routing	Independent stream per speaker
Volume	Software/stream volume; speaker's own stored volume stays unchanged
Bluetooth plan	Conservative design: 10 dedicated USB Bluetooth radios
Network	Ethernet preferred for Pi; phone can remain on Wi-Fi
Estimated hardware budget	Approximately \$290-\$450; practical target around \$350

1. Goal

Build one central device that replaces the separate MP3 players currently connected to multiple Bluetooth speakers.

Requirements: store all existing music files; connect by Bluetooth to multiple speakers; allow each speaker to play a different file simultaneously; control everything remotely from a phone; show speaker online/offline status; independently select tracks and volume; support custom web/mobile applications; accommodate YouTube Music where technically possible; scale to 10 speakers; and use a 500GB USB SSD.

2. Why the Tablet Idea Was Replaced

The original preference was an off-the-shelf tablet-style device. The problem is audio routing: ordinary Android tablets are not designed to route many different simultaneous audio streams to many different Bluetooth speakers.

Features such as dual Bluetooth audio generally focus on sharing the same media rather than providing 10 independently routed playback sessions. Linux provides much more direct control over Bluetooth controllers, audio routing, background services, and custom APIs.

Decision: use a Raspberry Pi/Linux system. After initial setup it should behave like an appliance rather than a computer: power on, boot automatically, reconnect speakers, mount the music library, start the audio services and web app, and require no monitor or keyboard during normal use.

3. Final Scale and Playback Model

The project started at approximately 5 speakers and was increased to **10 independent Bluetooth speakers**.

Each speaker can potentially play a completely different track at the same time. For example: Speaker 1 -> Track A, Speaker 2 -> Track B, continuing independently through Speaker 10 -> Track J.

4. Main Computer

The initial 5-speaker concept could likely have used a Raspberry Pi 4 with 2GB RAM. After increasing to 10 speakers, the selected target is a **Raspberry Pi 4 with 4GB RAM**.

The additional memory provides comfortable overhead for the Linux Bluetooth stack, PipeWire, 10 playback processes, the web backend, device state, music indexing, WebSockets, and other services. An 8GB model is unnecessary for the current design. A Pi 5 would work but is not required unless pricing makes it attractive.

5. Storage

Decision: 500GB USB SSD. It will hold the centralized music library, uploaded files, indexes/database information, and future content.

Use a separate 32GB or 64GB microSD card for Raspberry Pi OS. Prefer connecting the SSD directly to a Pi USB 3 port rather than sharing the Bluetooth hub.

6. Bluetooth Architecture

The Pi has built-in Bluetooth, but relying on one radio for 10 independent simultaneous A2DP streams is not the conservative approach.

Chosen safe architecture: approximately one dedicated USB Bluetooth radio per speaker - 10 identical USB Bluetooth adapters for 10 speakers. The built-in radio does not need to be used for speaker playback.

This makes controller-to-speaker assignment clearer, isolates connections, and simplifies troubleshooting and reconnection behavior. The exact adapter model remains to be verified for Linux/BlueZ support and reliable coexistence of 10 identical controllers.

7. USB and Powered Hub

Each Bluetooth adapter must ultimately connect by USB to the Raspberry Pi, but they do not need 10 separate built-in Pi ports. A single powered USB hub connects to the Pi, and the Bluetooth adapters connect to that hub.

Target: a powered 12-16 port USB hub with its own substantial AC power supply. This avoids asking the Raspberry Pi to power all radios and leaves expansion capacity.

Preferred topology: Pi USB 3 -> 500GB SSD; another Pi USB port -> powered hub -> 10 Bluetooth adapters.

8. Audio Software

The likely software stack is **BlueZ + PipeWire + WirePlumber**. BlueZ handles Bluetooth; PipeWire handles audio streams/routing; WirePlumber manages device/session policy.

The application can launch or manage separate playback sessions and route each session to its assigned Bluetooth output.

9. Volume Requirement

This requirement is explicit: changing volume in the web app should change the **audio signal being transmitted**, not the speaker's own remembered hardware volume.

Example: the physical speaker remains set at 80%, while the Pi digitally reduces its outgoing stream to 30%. Bluetooth absolute/hardware-volume synchronization should therefore be disabled or avoided where necessary, and stream/software volume should be used.

10. Web Application

The phone does not need to manage Bluetooth. It connects over Wi-Fi/network to the Raspberry Pi web application. The Pi handles Bluetooth connections and playback.

The dashboard should show each speaker's name, online/offline and connected status, current track, independent volume, and playback controls. Useful controls include song selection, play, pause, stop, restart, previous/next, reconnect, and potentially repeat/loop.

The music interface should support upload, browse/search, folders/categories, metadata, and storage information. System status can expose SSD free space, adapter health, speaker last-seen time, playback position, and current track.

11. Local Files and YouTube Music

Existing files from all separate MP3 players can be copied to the 500GB SSD, creating one centralized library from which the Pi creates 10 independent streams.

Local files and audio supplied by the custom application are straightforward. **YouTube Music is a separate integration question** because authentication, APIs, DRM/service restrictions, and official playback behavior need to be considered. It should not block the core local-file architecture.

12. Networking

Ethernet is strongly preferred for the Raspberry Pi. Bluetooth already operates in the 2.4GHz spectrum; keeping the Pi's network traffic wired reduces unnecessary wireless competition. If Ethernet is unavailable, 5GHz Wi-Fi is preferable to 2.4GHz Wi-Fi.

The phone can remain on Wi-Fi. If external/internet control is later required, a secure VPN or authenticated remote-access layer can be added.

13. RF / Reliability Risk

The main technical risk is not CPU or RAM. It is running **10 simultaneous Bluetooth A2DP streams in close physical proximity**.

Possible issues include 2.4GHz congestion, packet loss, range reduction, dropped connections, USB radio interference, controller assignment, and speaker reconnection behavior.

Mitigations include Ethernet, carefully selected Bluetooth chipsets, a quality powered hub, and short USB extension cables to physically separate radios if necessary.

14. Current Hardware Plan

Component	Current plan
Main computer	Raspberry Pi 4, 4GB
Music storage	500GB USB SSD
OS storage	32GB or 64GB microSD
Bluetooth	10 identical Linux-compatible USB Bluetooth adapters
USB expansion	Powered 12-16 port USB hub
Pi accessories	Proper USB-C power supply + cooled case
Network	Ethernet cable / wired connection preferred
Optional	Short USB extension cables for RF spacing

15. Estimated Hardware Cost

Component	Approximate
Raspberry Pi 4 4GB	\$60-\$100
500GB USB SSD	\$50-\$80
10 Bluetooth adapters	\$80-\$120
Powered 12-16 port USB hub	\$60-\$110
Power supply, case/cooling, microSD, misc.	~\$40
Estimated total	\$290-\$450
Practical target	Around \$350

16. Decisions Already Made

- One central device replaces the separate MP3 players.
- Raspberry Pi/Linux is preferred over an Android tablet.
- System is being designed for 10 independently playing speakers.
- Use a 500GB USB SSD for the music library.
- Use Raspberry Pi 4 with approximately 4GB RAM.
- Use a powered USB hub and, conservatively, one dedicated Bluetooth USB controller per speaker.
- Use software stream volume without changing each speaker's stored volume.
- Control everything through a custom web application from the phone.

- Phone communicates with the Pi over the network; Bluetooth is primarily Pi-to-speaker.
- Ethernet is preferred for the Pi.

17. Items Still to Finalize Before Ordering

Bluetooth adapters: Exact chipset/model must be verified for Raspberry Pi/Linux, BlueZ/A2DP source support, reliable operation of 10 identical controllers, availability and price.

Powered USB hub: Exact 12-16 port model with a strong external power supply and reliable USB implementation.

SSD: Exact 500GB portable SSD or SATA SSD + USB 3 enclosure, depending on value.

Pi source/accessories: Exact retailer, case/cooling, power supply and microSD based on current pricing.

18. Expected Finished Experience

At power-on, the Pi should automatically boot, mount the SSD, detect the USB Bluetooth controllers, reconnect paired speakers, start PipeWire and the backend/web interface, and expose the dashboard to the phone. Normal use should require no Linux interaction.

Next step: verify and select the exact Bluetooth dongle, powered USB hub, SSD, Pi, power supply, case, microSD, and any USB extension cables. The Bluetooth chipset/model deserves the most scrutiny because it is central to reliable 10-speaker operation.